QCon
全球软件开发大会

InfoQ 技术大会

从沙箱到执行边界：蚂蚁 AI Agent 的大规模企业级实践

林鑫

蚂蚁集团 高级技术专家



QCon
全球软件开发大会

InfoQ 技术大会

目录

01 行业趋势与项目背景

02 Sandboxed Agent：沙箱作为Agent的“家”

03 Agentic RL：沙箱作为大规模环境供应链

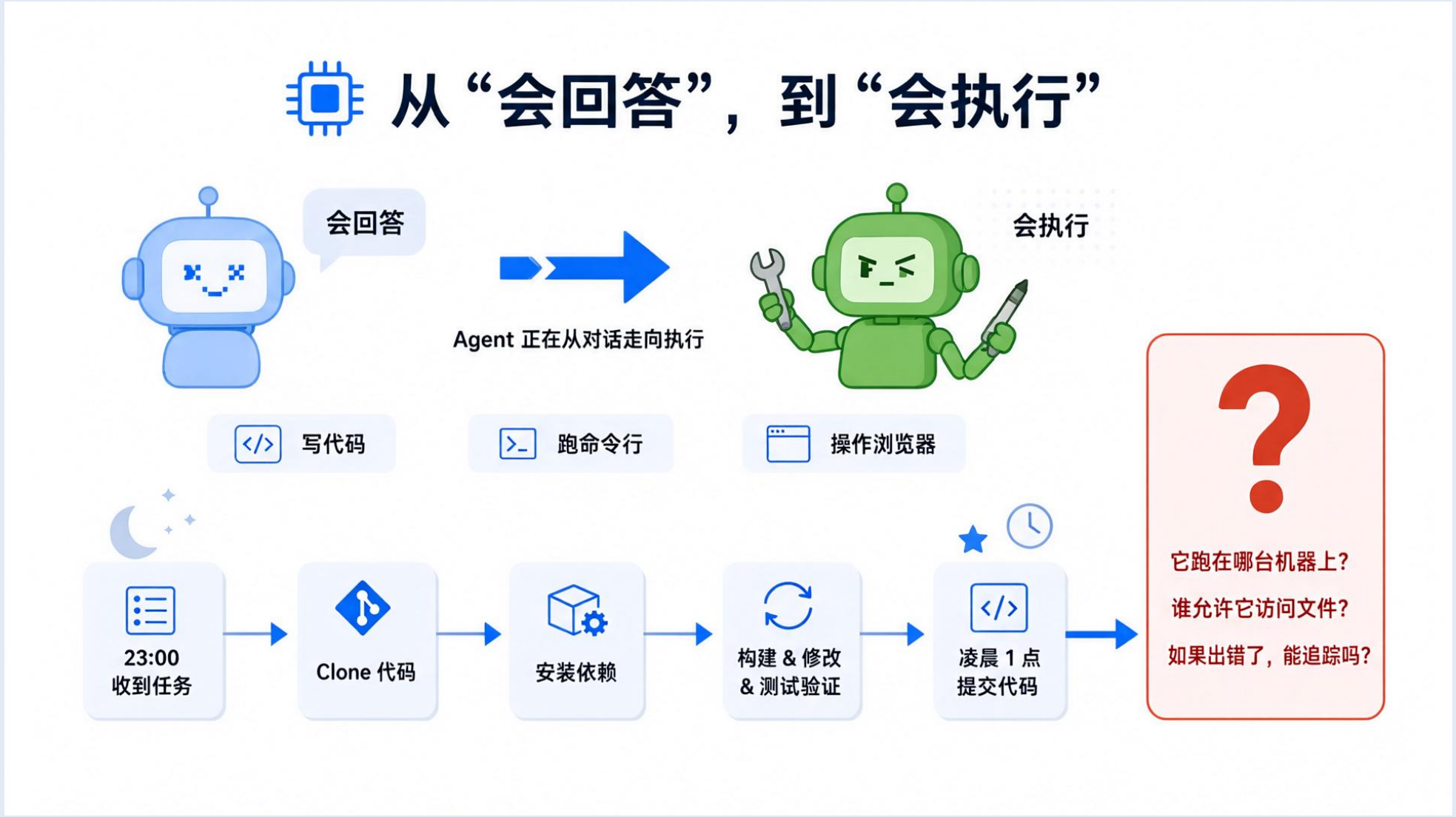
04 Agent Harness + Tool Use：沙箱作为高危动作执行器

05 总结

01

行业趋势与项目背景

Agent 从回答问题到执行动作



Agent 开始动手后，问题也变了

1 自主性 × 安全边界

Agent 自己决定下一步，还要碰各类数据；限制太死没法用，太松不敢放。

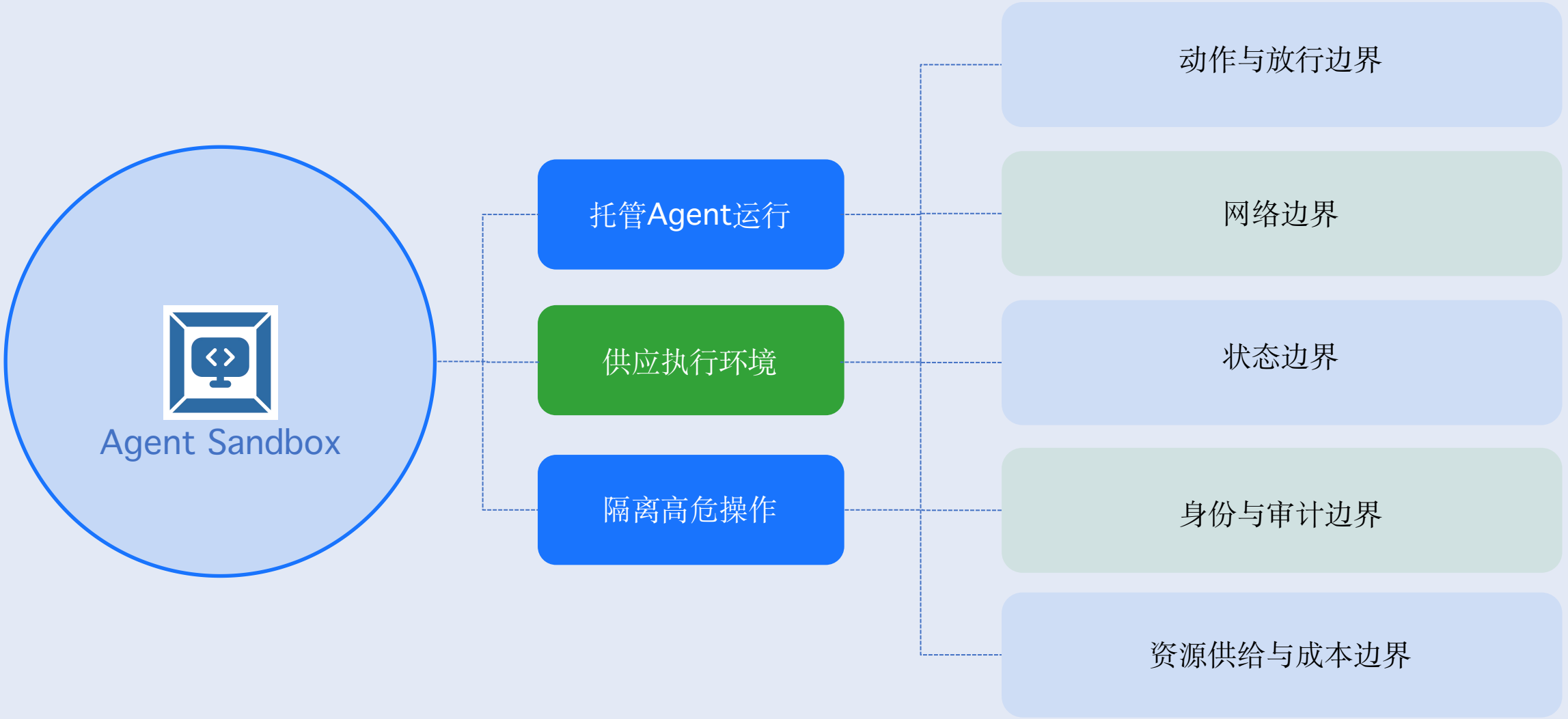
2 长生命周期 × 资源成本

Agent（例如OpenClaw）要长期运行，Workspace、会话、依赖都要留得住；资源却不能一直空转。

3 瞬时规模化 × 环境一致性

Agentic RL 要瞬间拉起几百上千个一致环境，镜像、依赖、缓存、调度一起承压。

执行边界：三类场景 x 五类边界



ARCA：把执行边界做成平台

ARCA：Ant Resilient Container Service for Agents

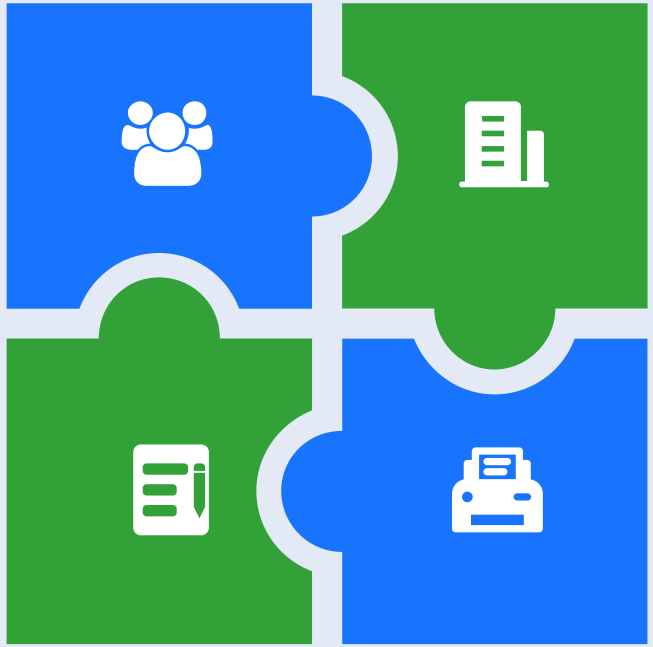
拉丁语“箱子、盒子、匣子”——一个能把珍贵之物稳定收纳、护在其中的容器。

百毫秒级别的沙箱创建延迟

通过沙箱缓存池、快照等技术，将沙箱创建的延迟降低到百毫秒的级别。
日均业务创建沙箱100w+，处理亿级别的沙箱请求。

资源利用率

通过超卖、外置持久化存储、checkpoint/restore等手段避免资源空转、降低资源使用成本。



多维度的安全隔离

基于rund安全容器的环境隔离、基于VPC 的网络隔离环境、租户和身份的隔离、存储隔离

100%的访问审计覆盖

通过出入口网关的管控，代理4层网络请求，100%进行沙箱访问请求的审计覆盖。

02

Sandboxed Agent：沙箱作为Agent的家

Agent 要住下来，平台要省成本

环境不丢失

持久化workspace、依赖不重装

任务要连续

会话不断线、支持长生命周期会话



低空转，资源充分利用

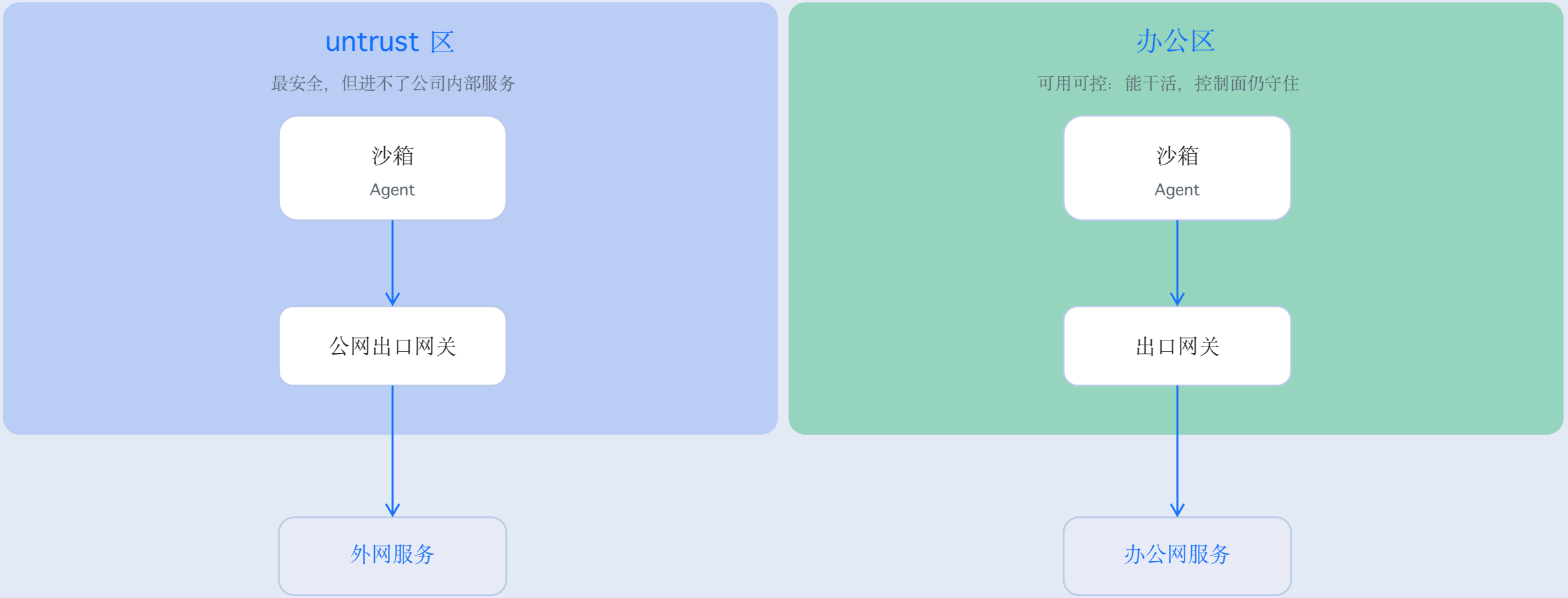
非活跃沙箱可暂停，资源可释放

可中断，系统能持续升级

长任务可做checkpoint，能按计划执行基础设施的升级

住下来 ⇄ 养得起

先让 Agent 住下来：从 untrust 区到办公区



边界不是越严越好，而是该放的执行放开、该守的管控守住。

状态拆开治理：实例可以回收，状态都还在

计算状态

进程 / 运行时 / 内存
实例可回收、可重建

身份状态

独立注入、随任务生命周期
不绑进程 / 实例



文件状态

workspace / 配置 / 上传文件
落持久化，跨实例复用

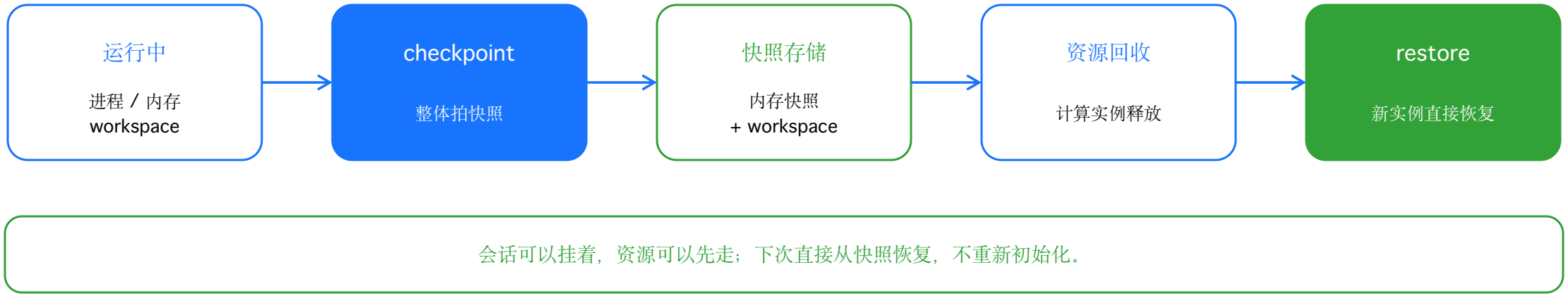
成本

资源独立计量
按任务归因、可控制

checkpoint / restore：会话可以挂着，资源可以先走

把运行中的环境整体保存，下次直接恢复，不重新初始化

生命周期：运行 → 快照 → 回收 → 恢复



快照什么

进程、磁盘

怎么恢复

新实例直接恢复
不重新初始化

会话不丢

任务暂停，上下文不断
可以接着执行

资源可走

计算实例释放
成本可以下降

把能力做成产品：业务只管 Agent，平台负责托管

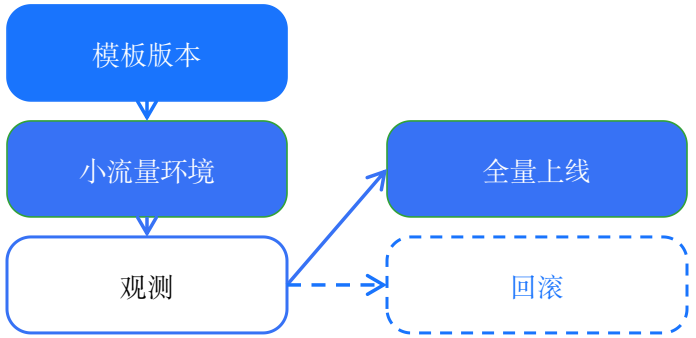
五项托管能力

- 模板
- 生命周期托管
- 灰度发布
- 自助入驻
- 运维监管

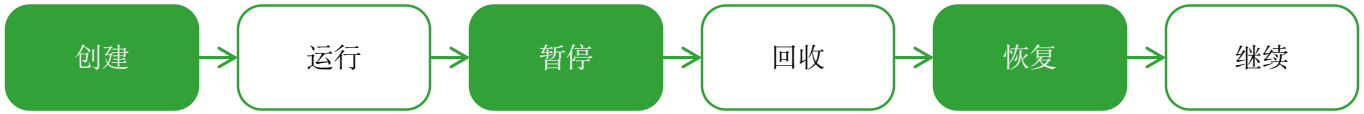
重点一：模板能力 + 基于模板版本的灰度发布

模板能力

环境定义 / 依赖与工具链
资源规格 / 网络与权限
配置制品化 / 版本管理



重点二：生命周期托管



生命周期由平台托管；checkpoint / workspace / 会话上下文随任务保留

模板定义环境，模板版本驱动灰度发布；生命周期托管让 Agent 一直可用。

执行底座：让不同 Agent 场景跑起来

五类沙箱，对应不同 Agent 场景

代码解释沙箱

代码执行 / 构建测试
数据分析 / 依赖安装

浏览器沙箱

网页操作 / 信息提取
端到端测试

电脑沙箱

桌面 GUI / 软件操作
复杂自动化

移动沙箱

App 自动化
移动端测试

自定义沙箱

业务专属环境
特殊依赖与工具链

统一沙箱抽象

统一接口：创建 / 执行 / 文件 / 快照 / 恢复 / 状态挂载
业务不用关心底层是哪一类沙箱
新场景用自定义沙箱扩展，而不是重做一套底座

安全容器运行时

隔离与资源约束
支持快照、恢复、状态挂载
保障沙箱稳定运行

不同 Agent 场景，都能在同一套执行底座上跑起来。

成本治理：通用手段 + 平台重点

CPU 超卖

提高单机资源利用率
关键场景不参与超卖

空闲回收

非活跃实例回收计算
状态保留、任务可恢复

业务实测（预热池 → 缓存池）

成本 -96%

启动快了，不再长期占着 XX 万个预热实例

从预热池到缓存池

早期：预热池

提前启动实例

命中时启动快

代价：空闲资源占用

容量难预估

模板版本多时覆盖困难

→

现在：缓存池

缓存镜像 / 依赖 / 内存快照

按需创建、快速恢复

容量可控

利用率更高

缓存池 + 快照：既控制平台成本，同时也提升沙箱的启动效率

通用的 CPU 超卖、空闲回收是通用能力 + 缓存池能力，控制Agent运行成本

生产风险：共享存储的越权修改问题

① 风险是怎么发生的

沙箱 A

Agent · root 启动

沙箱 B

Agent · root 启动

沙箱 ...N

Agent · root 启动

挂载
（预期只读）

① root 绕过 555：沙箱可写 NAS

NAS（共享存储）

系统 Skill
系统级能力，被 Agent 加载执行

权限管控：555（只读位）

同一份 Skill 被多个沙箱挂载：任意一个沙箱改写，其他沙箱下次加载都会拿到被改过的内容。

② 风险为什么会扩散

这个业务有多个沙箱，它们挂载的是同一个 NAS 目录。

一个沙箱里的 Agent 用 root 改写 Skill 后，其他沙箱不会察觉——下次加载时拿到的是被改过的内容，行为被同步改变。

所以这不是一个沙箱的故障，而是这个业务所有沙箱共担的风险。

③ 我们的修复

沙箱侧：改成只读挂载（ro）

写请求在挂载点就被内核拒绝（EROFS）。只读是挂载的属性、不是文件的属性，root 也绕不过。

业务侧：保留可写挂载（rw）

Skill 的更新只来自业务管控平台，写入路径唯一、可审计；写权限单独授予，沙箱侧一律只读。

权限管控不能停在文件权限位：共享资源的写权限要单独授予

谁能写、以什么身份写、写进去影响谁——要在边界上收住，而不是靠一个 555 权限位

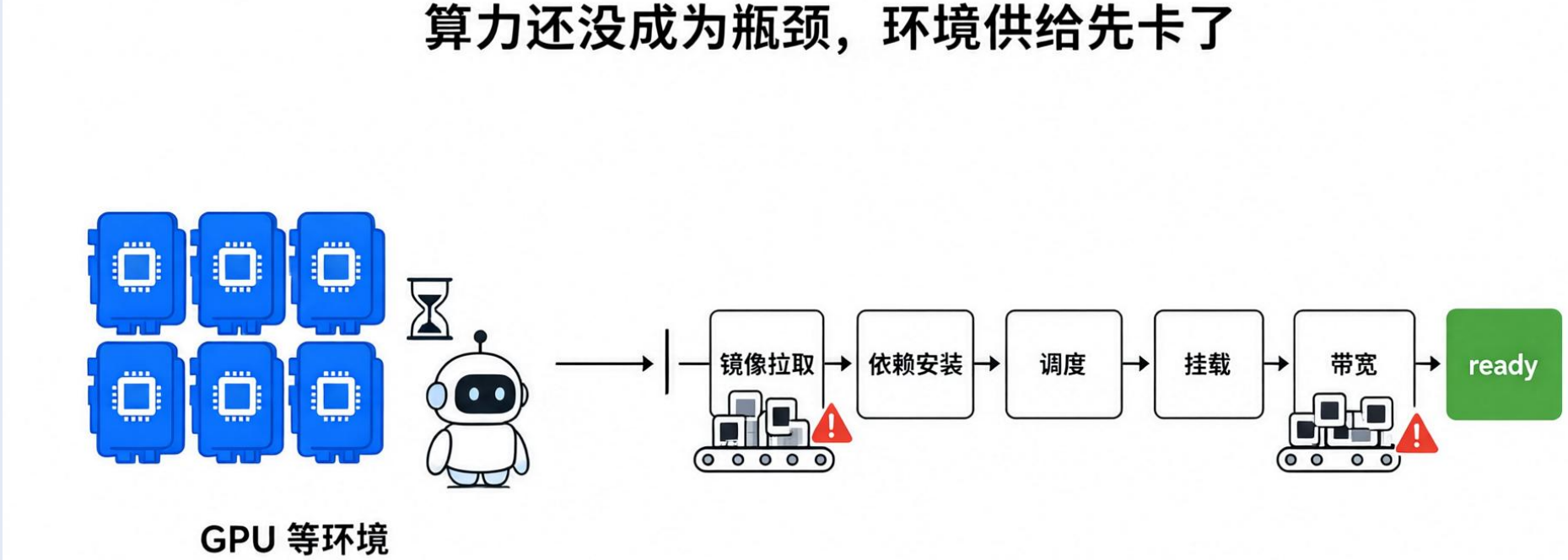
让 Agent 住的下来，也养得起



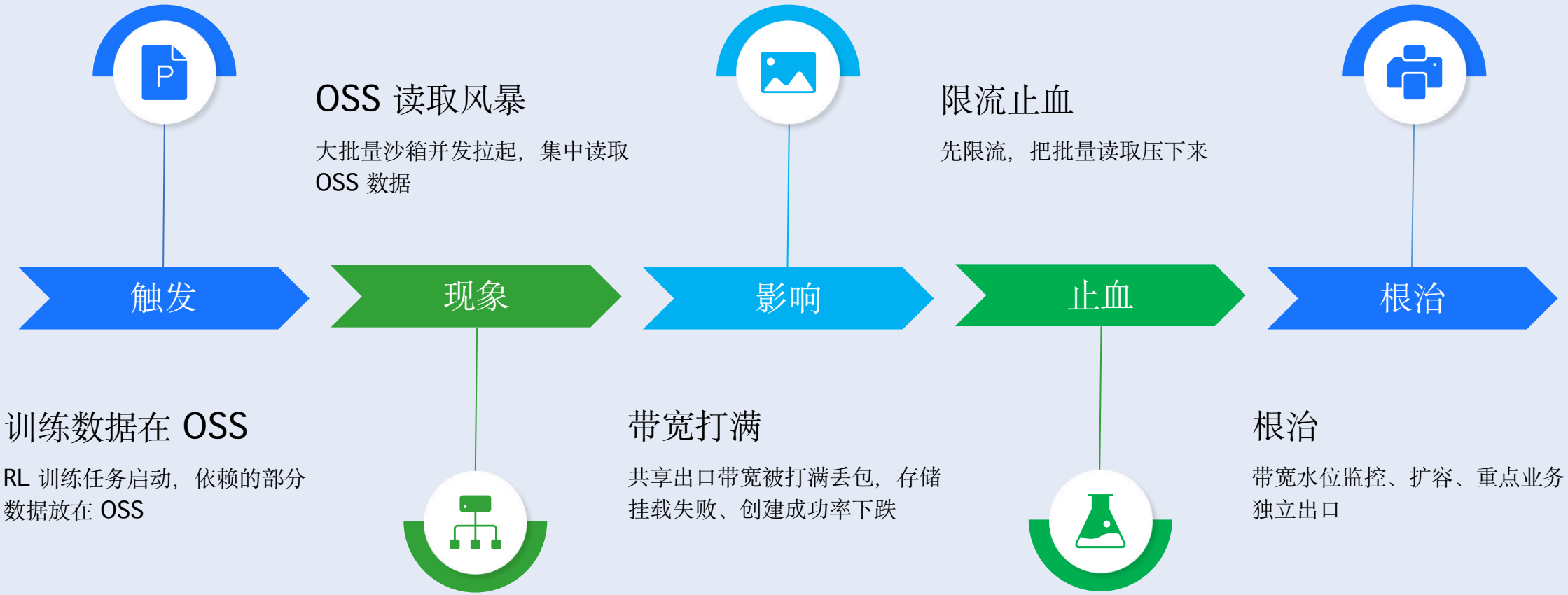
03

Agentic RL：沙箱作为大规模环境供应链

瓶颈不只是算力



一次真实事件：大批量沙箱集中读取 OSS，共享出口带宽先被打满



拆冷启动：把串行初始化拆成可缓存、可并行的路径

Nydus 镜像加速 · CRIU 内存快照 · 缓存池

启动路径：先拆耗时，再分冷热启动

镜像拉取 | 依赖 / 初始化 | 运行时启动 | 交付

串行路径：每一步都依赖上一步，任何一步慢都会拖住 ready

热启，命中路径：缓存池命中 → 直接分配就绪沙箱 → ready

冷启，未命中路径：Nydus 按需加载 → 快照恢复 / 冷启动 → ready

缓存池

机制
- 预创建就绪沙箱，按模板版本维护
- 用一个补一个；未命中走冷启动
- 容量 = 创建 QPS × 启动耗时

收益：命中时直接分配，端到端更短

代价：占租户配额 / 计费 / 灰度新老双池

Nydus 镜像加速

机制
- 镜像层缓存预热 + 按需加载
- blob cache / page cache / P2P
- Nydus / OCI 回退 + 最优节点调度

收益：分钟级拉取 → 秒级启动

代价：首次无缓存慢 / miss 惩罚 / 兼容回退

内存快照（CRIU）

机制
- CRIU 全量快照：进程 / FD / 内存 / 网络
- pause 释放计算，restore 保持 sandboxID
- 一份快照 fork 多个沙箱

收益：命中直接恢复，跳过初始化

代价：全量快照大 / 镜像升级不兼容 / 连接短暂中断

冷启动优化 = 镜像加速 + 快照恢复

缓存感知调度：让沙箱落到已经准备好的节点

调度输入

资源容量 / Region / 集群 / 镜像缓存 / 快照 /优先级

通用节点的降级

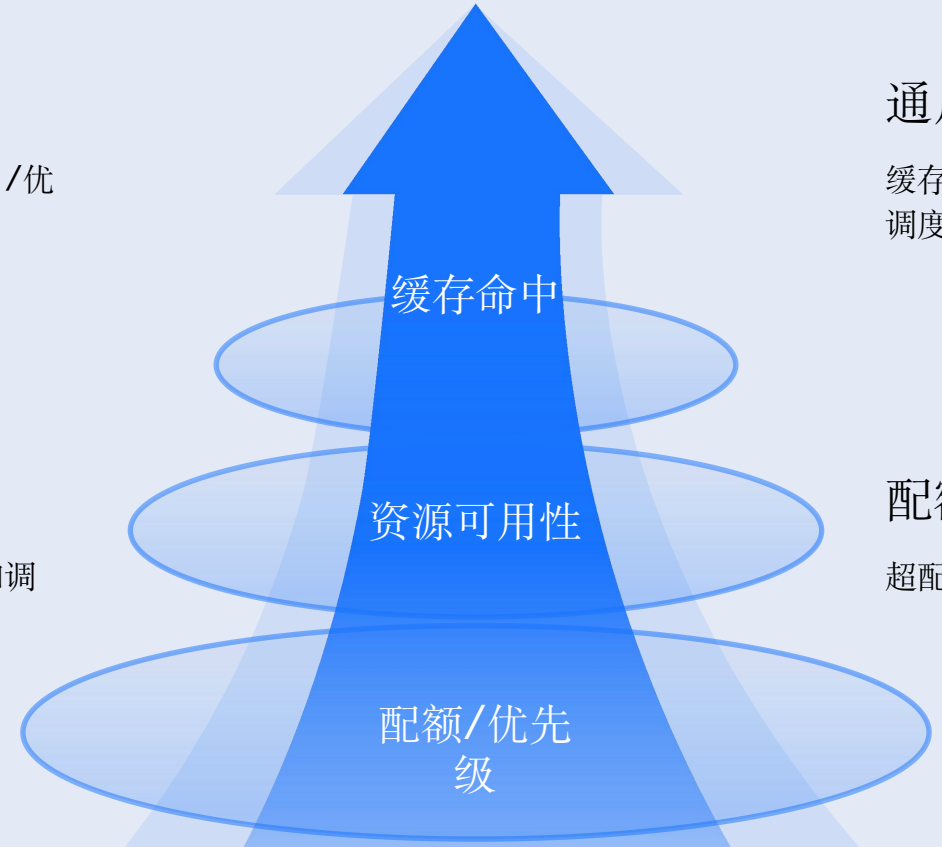
缓存不命中 + 资源满足 + 配额满足 → 没有亲和调度，降级到通用调度

命中预热节点

缓存命中 + 资源满足 + 配额满足 → 采用亲和调度，调度到最快Ready的节点

配额拒绝

超配额 / 资源不足 → 拒绝



在配额允许、资源足够的前提下，找一个最快ready的节点

可观测：拆分环境供给的各个阶段进行度量

五段各自的指标与失败归因

资源准备

大规模镜像拉取

指标：并发拉取数、峰值拉取带宽、镜像拉取延迟、预热成功率、缓存命中率等

失败归因：镜像仓库不可用、镜像缺失、拉取超时、带宽打满

调度

大量的沙箱调度

指标：创建沙箱QPS、调度耗时、亲和命中率、缓存池命中率、缓存实例消耗和补充速率等

失败归因：资源不足、配额不足、超时等异常

挂载

存储挂载

指标：挂载耗时、挂载成功率、存储使用量等

失败归因：权限、限流、超时等

启动

业务启动

指标：镜像大小、业务启动耗时、快照恢复时长等

失败归因：依赖缺失、业务错误、超时等

就绪

端到端的交付

指标：端到端交付耗时、创建成功率、释放成功率、释放耗时、活跃沙箱数、活跃时长等

失败归因：明确各类错误码，区分业务错误还是平台错误

环境供给：从瓶颈到能力

把环境做成一条可管理的供应链

可复现	环境不是变量：镜像 Digest + 模板版本 + 依赖锁定
可加速	通过多种缓存、分层等机制，提升交付速率
可调度	先准入、再过滤、最后优选：让沙箱落到已经准备好的节点
可观测	五段指标加 traceID：失败能定位到哪一段、哪一类原因

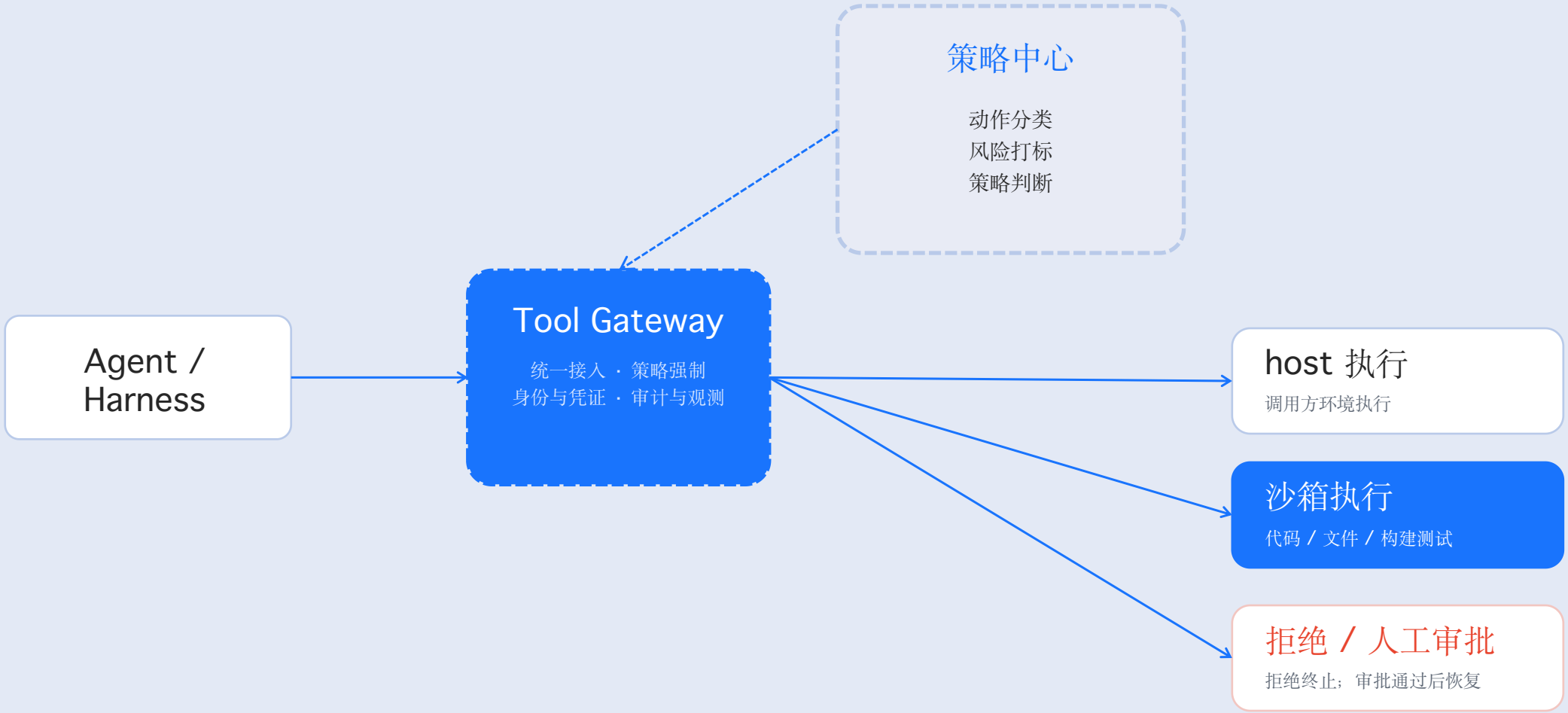
04

Agent Harness + Tool Use：沙箱作为高危动作执行器

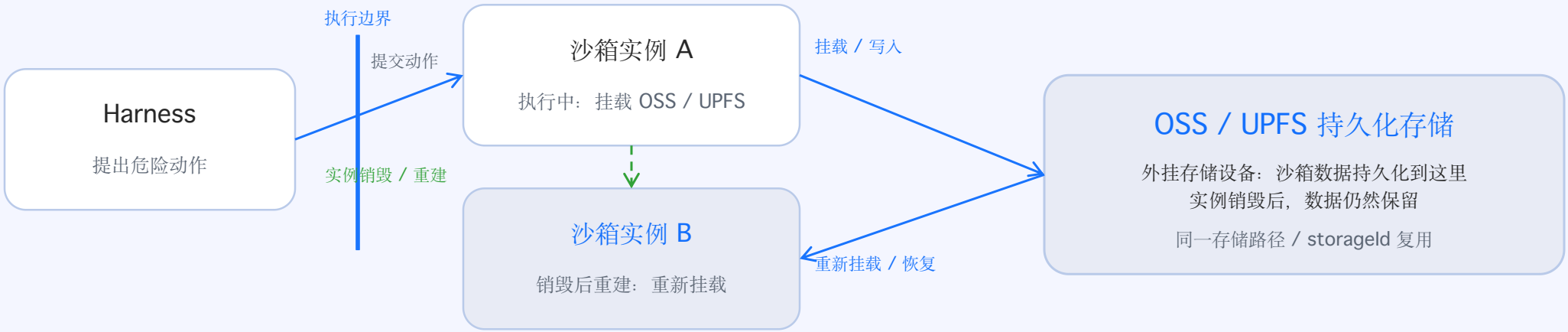
沙箱作为高危动作执行环境



运行时策略：先判断，再放行



状态共享：沙箱销毁，但状态可以保留



沙箱 A 挂载 OSS / UPFS → 执行并写入数据 → 销毁实例 → 创建沙箱 B → 重新挂载同一存储 → 继续执行 / 重试

外挂存储

OSS / UPFS 作为沙箱外部
持久化设备

挂载写入

沙箱启动挂载
执行过程写入数据

实例销毁

计算实例回收
数据仍在外部存储

重建恢复

新沙箱挂载同一存储
继续执行 / 重试

实例可以销毁，数据留在 OSS / UPFS；新实例挂载回来就能继续。

身份与审计：每一次执行都可归因

证据链：从请求到审计，trace id 全程贯穿

1	2	3	4	5
请求方	身份签发	沙箱执行	出口网关	全链路审计
用户 / Agent / 租户	短期委托身份 / 最小权限 token	唯一身份，可验签	目标白名单 / 请求记录	trace id / 调用链 / 结果

谁发起？

用户 / Agent / 租户身份

以谁的身份执行？

短期委托身份 / 最小权限 token

访问了什么？

目标资源 / 出口网关记录

结果是什么？

trace / 调用链 / 操作审计

审计证据链

请求 → 策略决策 → 身份签发
沙箱执行 → 出口访问 → 结果

trace id 全程贯穿，支持回放与追责

出事了不是翻日志，而是把身份、决策、调用链串成一条证据链。

本章总结：危险动作进受控边界

运行时策略

动作分类 → 风险打标 → 放行前判断

Tool Gateway

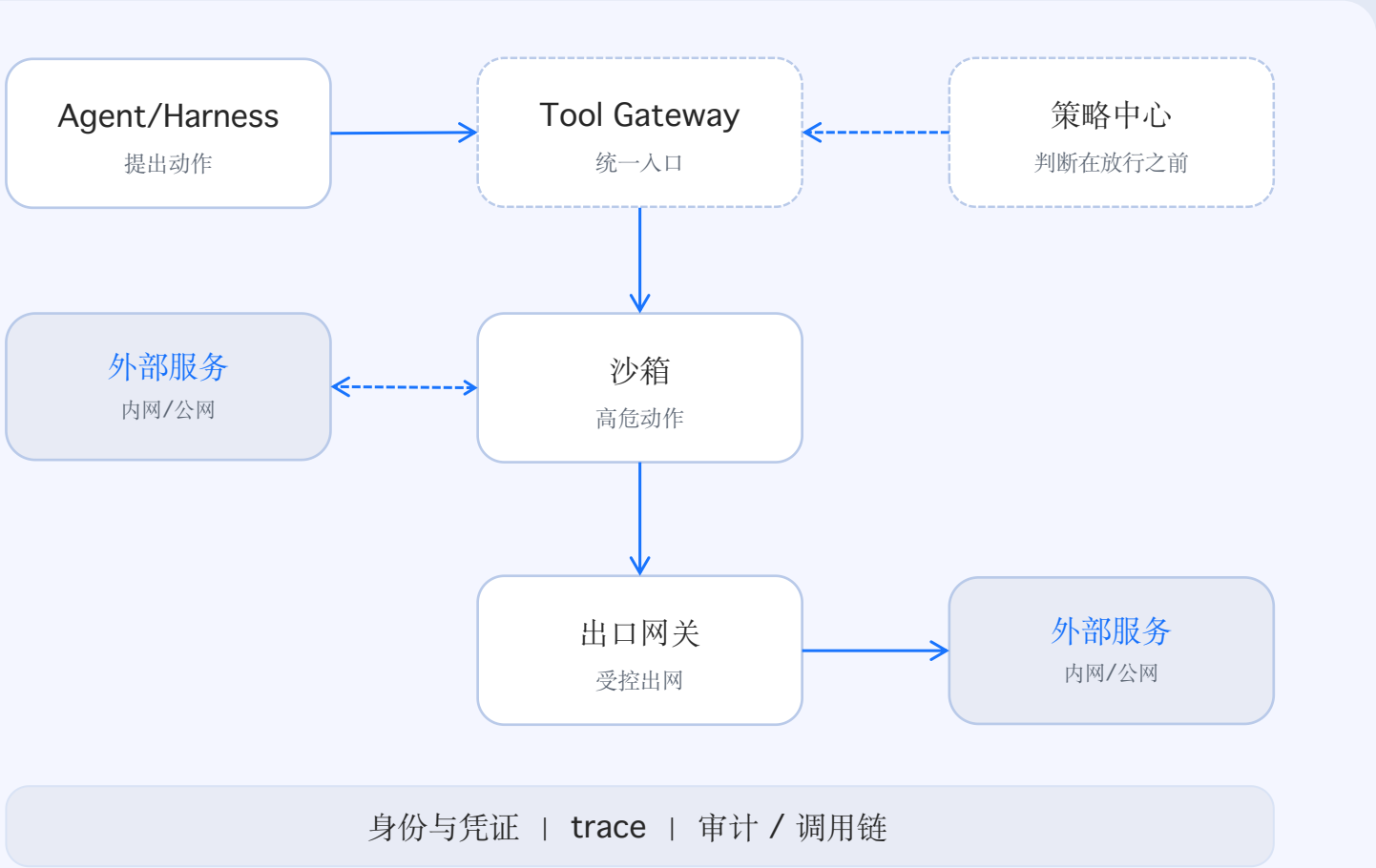
工具调用统一入口，按策略强制分发

状态共享

沙箱外挂OSS/NAS/UPFS，实例重建可恢复状态

身份与审计

短期委托身份、最小权限Token、Trace全链路追踪

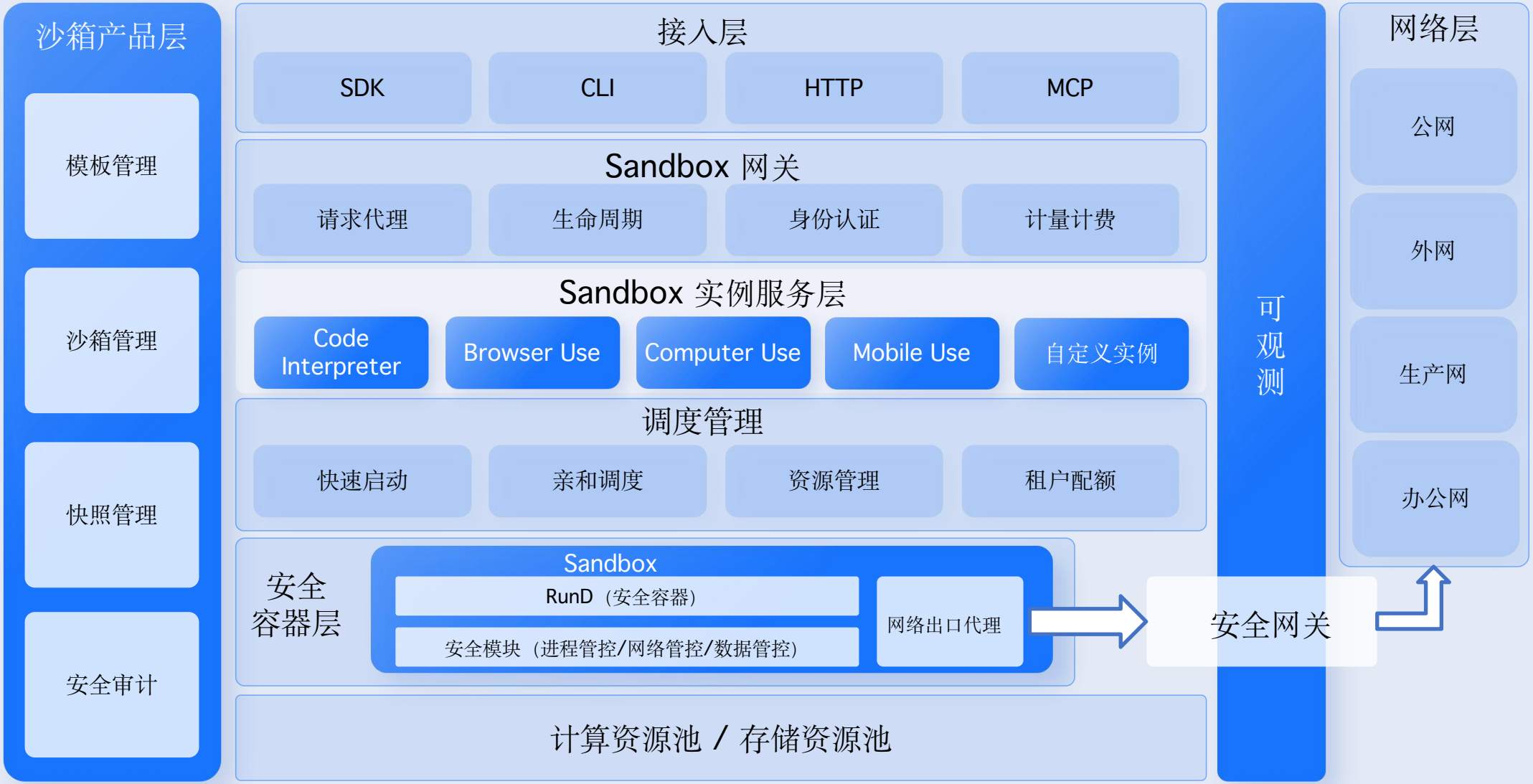


不是所有东西都进沙箱，而是把危险 Action 路由进受控执行边界。

05

总结

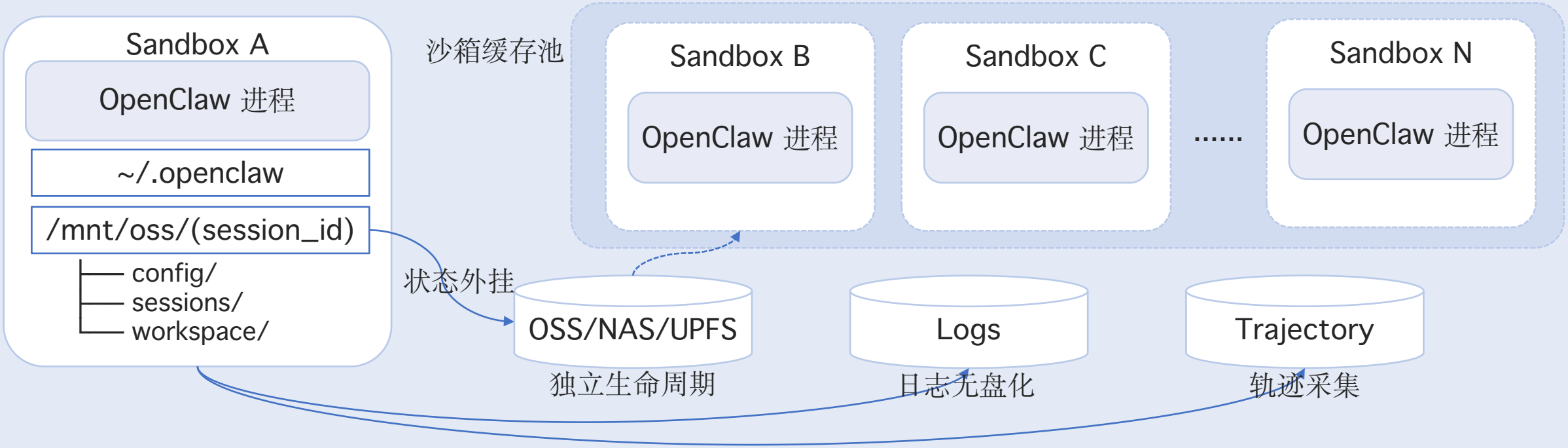
三类场景，一套架构


沙箱产品层
模板管理
沙箱管理
快照管理
安全审计
接入层
SDK
CLI
HTTP
MCP
Sandbox 网关
请求代理
生命周期
身份认证
计量计费
Sandbox 实例服务层
Code Interpreter
Browser Use
Computer Use
Mobile Use
自定义实例
调度管理
快速启动
亲和调度
资源管理
租户配额
安全容器层
Sandbox
RunD（安全容器）
安全模块（进程管控/网络管控/数据管控）
网络出口代理
计算资源池 / 存储资源池
可观测
安全网关
网络层
公网
外网
生产网
办公网

满足三类场景的关键点

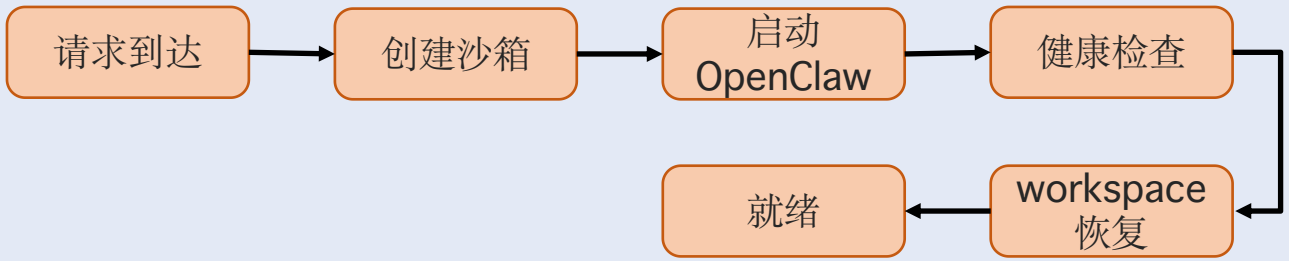
场景不同，核心要求不同			
对比维度	Agent的家	环境供应链	高危动作执行器
典型形态	托管 Agent 运行	RL训练与评测	Agent 工具调用
核心诉求	环境不丢、会话不断、成本可控	批量供应、可复现	动作执行风险管控
架构落脚点	生命周期、持久化存储、审计	快照、吞吐	安全容器、极速启动
关键做法	空闲资源回收、沙箱轮转	亲和调度、镜像加速、轨迹	沙箱缓存池
共同点	安全隔离、网络访问、状态、身份、成本、可观测诉求		

托管OpenClaw的实践案例



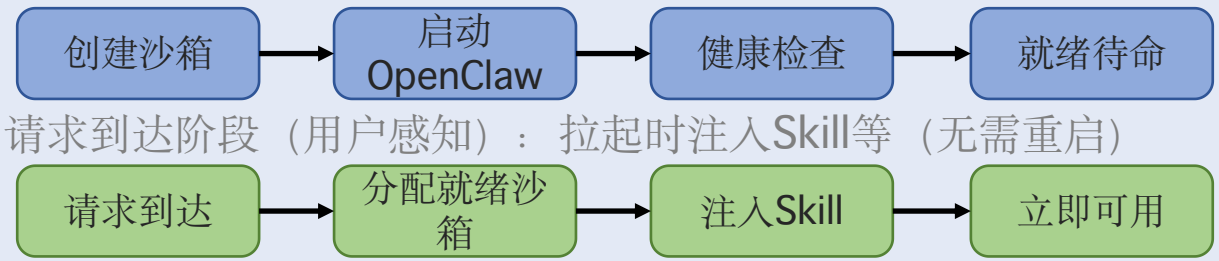
优化前：冷启动 5s+

用户全程等待



优化后：缓存池&热加载 1s+

预热阶段（提前完成）：预热时固化Gateway、Plugin等



请求到达阶段（用户感知）：拉起时注入Skill等（无需重启）

把危险关进边界，把能力留给 Agent

从 Demo 到规模化落地

执行边界不是一堵墙，是一组受控点

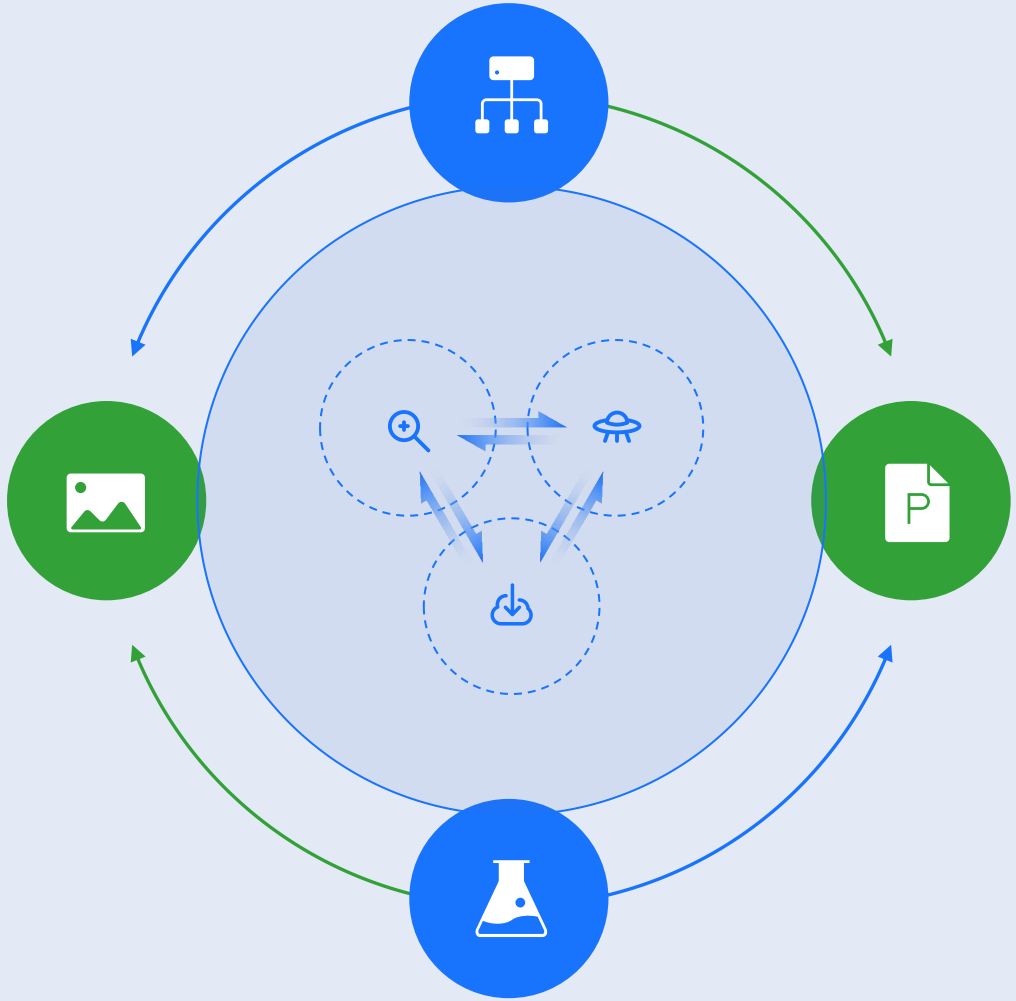
动作、网络、状态、身份、成本，五类边界分别收口

同一套沙箱底座，三种用法

Agent的托管、大规模环境的供应、高危动作执行器

把边界做成平台能力，才能支持规模化

可复现、可加速、可审计、可观测



QCon
全球软件开发大会

InfoQ 技术大会

THANKS



AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师